CSS4003 · DevOps 2 — Final Project: "My Service in Production"

Weight: 40% of the course grade (the final exam: project with defence). **Format:** individual. Every student gets a personal **variant**. **Deadline:** any day before the final exam, **including the final exam day**. You can submit and defend early.

You take a small web application from our template and build everything a real team needs to run it in production: Linux operations, a Git workflow, a cloud network, containers, Kubernetes, a self-hosted database with roles and backups, secrets in a cloud secret manager, security checks, monitoring, central logging, CI/CD and Terraform. When you're done, you have a public repository you can show to an employer.

> **The app is given; the platform is yours.** You may change the app (add an endpoint, a field, a page), but the grade is about everything around it.

---

1. What you start from

Create your own **public** repository from the template: `weeebdev-edu/css4003-final-project` → **"Use this template"**. Name it `<your-prefix>-campus-platform` (your prefix is in your variant).

The template contains `campus-app`, a FastAPI service:

Feature	How it works
Configuration	Only environment variables (12-factor)
Database	PostgreSQL **or** MySQL via `DB_URL`
Analytics (optional)	ClickHouse via `CLICKHOUSE_URL`
File storage	`STORAGE_BACKEND=local|s3|gcs`
Health	`GET /healthz` (liveness), `GET /readyz` (checks the DB)
Metrics	`GET /metrics` (Prometheus format)
Logs	JSON to stdout, one line per request, with a `request_id`
Secrets	Every secret can be read from a file: `DB_PASSWORD_FILE`, `SECRET_KEY_FILE`, …
Security	`SECRET_KEY` must be ≥ 32 characters; `POST /api/admin/check-password` checks a password against the policy
DB roles	`sql/roles.<engine>.sql` with example `admin`, `app_rw`, `app_ro` roles
Container	Multi-stage, non-root `Dockerfile`; `compose.yaml` for local development
CI	A minimal GitHub Actions workflow: lint + tests

2. Learning goals → syllabus

Syllabus week	What the project proves you can do
1–3 Linux, processes, services, networking	Write and schedule ops scripts (backup, restore, health check); debug with `ss`, `curl`, `journalctl`, `kubectl logs`
4–5 Git & GitHub, Advanced Git	Issues → branches → PRs with review → protected `main` → semver tags and Releases
6 Intro to Cloud	Work as a non-root identity, budget/alert, CLI profiles
7 EC2 / Compute Engine	VMs with startup scripts, key-based access, lifecycle
8 Networking	Your own VPC, public/private subnets, least-privilege firewall rules, DB never public
9 Storage & Docker fundamentals	Object storage, backups, multi-stage non-root images, compose
10 Docker + Cloud	Images in ECR / Artifact Registry, tags vs digests, no keys on servers
11–12 CI/CD	Pipelines with tests, security gates, image publishing, deployment, rollback
13 Terraform	The whole platform as code: `apply` from zero, `destroy` at the end
+ this project	Kubernetes orchestration, monitoring, logging, secret management, DB administration

---

3. Tracks: AWS or GCP

Pick the cloud where you have credits. Requirements and grading are the same.

Need	AWS	GCP
Network	VPC, subnets, route tables, Security Groups / NACLs	VPC network, subnets, firewall rules, Cloud NAT
VMs	EC2 (+ user data)	Compute Engine (+ startup script)
Kubernetes	**k3s on 1–2 EC2 VMs** (default) or EKS if you have credits	**k3s on 1–2 VMs** (default) or GKE if you have credits
Object storage	S3	Cloud Storage
Container registry	ECR	Artifact Registry
Secrets	Secrets Manager or SSM Parameter Store `SecureString`	Secret Manager
Identity for workloads	Instance profile / IAM role	Service account attached to the VM
CI → cloud auth	OIDC role, or a **self-hosted runner on your VM** using the instance role	Workload Identity Federation, or a self-hosted runner with the VM's service account
Cloud logs (`cloud-logging-fluentbit`)	CloudWatch Logs	Cloud Logging
IaC	Terraform `aws` provider	Terraform `google` provider

How to get free credits and stay inside them: **FREE_TIER_GUIDE.md**.

AWS Academy Learner Lab counts as the AWS track. Its limits (check the Learner Lab Readme):

- region `us-east-1`; sessions expire after a few hours, so credentials rotate;
- you **cannot create IAM users or roles** — use `LabRole` / `LabInstanceProfile`;
- OIDC from GitHub needs a role you can't create — use a **self-hosted GitHub Actions runner on your VM** (it gets credentials from `LabInstanceProfile`), or paste short-lived credentials into GitHub secrets right before a deploy and document it as a known limitation;
- credits are limited: destroy what you don't need.

---

4. Your variant

The instructor gives every student a personal variant. **See your variant from the instructor** and put it in the table at the top of your report. Graders check it.

Field	Values	Where it must show up
`theme`	your domain, e.g. "Library books", "Club events", "Dorm repairs"	`APP_NAME`, item names/fields in the app, README title
`resource_prefix`	e.g. `dv-ivanov-8299`	the start of **every** cloud and Kubernetes resource name
`vpc_cidr`	`10.X.0.0/16`	your VPC and subnets (Terraform)
`db_engine`	`postgresql` | `mysql`	the self-hosted database you run
`clickhouse_required`	`yes` | `no`	if `yes`: ClickHouse runs and receives analytics events
`cluster`	`k3s-1-node` | `k3s-2-nodes` | `managed-k8s-if-credits`	your Kubernetes cluster (see 5.5)
`logging_stack`	`loki-alloy` | `cloud-logging-fluentbit`	your central logging (see 5.10)
`password_min_length`	12–16	DB password policy and the CI password check
`password_require_symbol`	`yes` | `no`	same
`hpa_cpu_target`	50 | 60 | 70 (%)	your `HorizontalPodAutoscaler`
`alert_error_rate_pct`	1–5	your 5xx error-rate alert rule
`alert_p95_ms`	300–800	your latency (p95) alert rule
`backup_cron_utc`	e.g. `15 2 * * *`	your backup CronJob/timer schedule
`backup_retention_days`	7 | 14 | 30	lifecycle rule on the backup bucket

You must **show** each value applied: the prefix in resource names, the CIDR in the VPC, the thresholds in alert rules and the HPA, the policy in the DB and in the CI check, the schedule and retention in the backup setup. A project built with someone else's variant gets **0 for the report**.

---

5. Required components and acceptance criteria

Each block lists what must be **true and shown**. The report proves it (screenshots, command output, links to files/commits/pipeline runs).

5.1 Linux operations

- □ `ops/backup.sh`, `ops/restore.sh`, `ops/healthcheck.sh` with `set -euo pipefail`, usage messages and non-zero exit codes on failure
- □ Scheduled backups: Kubernetes `CronJob` (or cron/systemd timer on the DB VM) on **your variant's schedule**

□ VM bootstrap is automated (user data / startup script / cloud-init), not typed by hand
□ A short troubleshooting section in the README: which commands you use when the app is down

5.2 Git workflow

□ `main` is protected: PR required, at least one passing check required, no force-push
□ Work goes through Issues → branches → PRs (at least **8 merged PRs** over time, not in one evening)
□ Conventional commits (`feat:`, `fix:`, `ci:`, …)
□ At least **3 semver tags** with GitHub Releases and a `CHANGELOG.md`
□ `.gitignore` covers state files, `.env`, keys; **gitleaks finds nothing in the whole history**

5.3 Cloud & network

□ Your own VPC with **your CIDR**: at least one public and one private subnet
□ Only the ingress/load balancer is reachable from the internet (80/443). SSH only from your IP (or via SSM/IAP — no SSH at all is even better)
□ The database is in the private part and **not reachable from the internet** — prove it with a failed connection attempt from your laptop
□ Nodes use an instance role / service account — **no cloud keys on any VM or in any pod**

5.4 Docker & registry

□ Multi-stage Dockerfile, non-root user, final image small (report the size)
□ Images in ECR / Artifact Registry, tagged with the **git SHA** and the **semver tag**; never deploy `latest`
□ `compose.yaml` still works for local development

5.5 Orchestration (Kubernetes cluster)

□ A **cluster** as in your `cluster` variant, built on cloud VMs by Terraform/scripts:
- `k3s-2-nodes`: one server + one agent VM (a real multi-node cluster);
- `k3s-1-node`: one server VM (still provisioned and joined like a cluster; adding a second node is a plus);
- `managed-k8s-if-credits`: EKS/GKE with ≥ 2 nodes if your credits allow — otherwise fall back to `k3s-2-nodes` and say so in the report

□ **Node failure drill** (`k3s-2-nodes` and managed): `kubectl cordon` + `drain` a node, or stop the agent VM → pods are rescheduled to the other node and the app keeps answering
□ App runs as a `Deployment` with ≥ 2 replicas, `resources.requests/limits`, liveness on `/healthz`, readiness on `/readyz`
□ `Service` + `Ingress` (Traefik in k3s is fine) — the app is reachable by a URL
□ `HorizontalPodAutoscaler` on CPU with your `hpa_cpu_target`; show it scaling under load (`hey`, `k6`, `ab` or a loop)
□ Configuration in `ConfigMap`s, secrets in `Secret`s (delivered from the cloud secret manager — see 5.7)
□ Database as a `StatefulSet` with a `PersistentVolumeClaim` (or on its own VM in the private subnet)

□ Rolling update with zero failed requests, and a **rollback** (`kubectl rollout undo` or redeploying the previous tag) shown
□ Manifests in git (`k8s/` plain YAML, Kustomize or Helm — your choice)

5.6 Database: self-hosted, roles, backups

□ **Your variant's engine**, self-hosted. **Managed cloud databases (RDS, Cloud SQL, …) are not allowed.**
□ Roles: `admin` (DDL), `app_rw` (only DML on the app's tables), `app_ro` (only `SELECT`). The app connects as `app_rw`; reporting/Grafana uses `app_ro`
□ Prove least privilege: `app_ro` can't `INSERT`, `app_rw` can't `DROP TABLE`
□ Password policy (`password_min_length`, `password_require_symbol`) enforced **in the database** where the engine supports it (MySQL `validate_password`; PostgreSQL `passwordcheck` or `scram-sha-256` + your check script) **and** checked by the app endpoint `/api/admin/check-password`
□ Backups go to object storage on `backup_cron_utc`, old ones expire after `backup_retention_days` (lifecycle rule)
□ **A tested restore**: drop data → restore from the latest backup → data is back (show it)
□ If your variant requires ClickHouse: the app writes analytics events there, and one dashboard panel or query reads them

5.7 Secret management

□ All secrets (DB passwords, `SECRET_KEY`, any tokens) are stored in **Secrets Manager / SSM SecureString / GCP Secret Manager**
□ They reach pods through **External Secrets Operator** or a sync step (script/CI job) that creates Kubernetes `Secret`s; the app reads them via `*_FILE` or env
□ Nothing secret in git, images, Terraform state committed to git, or CI logs
□ Rotation: change one password in the secret manager → it reaches the app → show it still works

5.8 Security checks (CI gates)

Every PR runs these, and a failure **blocks the merge**:

□ **gitleaks** — secret scan
□ **Trivy image scan** — fails on `HIGH`/`CRITICAL` (document any accepted exceptions with a reason)
□ **pip-audit** (or equivalent) — dependency audit
□ **IaC scan** — `trivy config`, `tfsec` or `checkov` on your Terraform and manifests
□ **Password policy check** — a job that calls `/api/admin/check-password` (or a script) with weak and strong examples from your variant and fails if a weak one passes
□ Show one PR that was **blocked** by a gate, and the fix

5.9 Monitoring

□ Prometheus scrapes the app's `/metrics`, node metrics and the database (exporter)
□ A Grafana dashboard: **RED** (rate, errors, duration) for the app, pods/restarts, node CPU/memory, DB health — exported as JSON into the repo
□ **≥ 3 alert rules**, including 5xx error rate > `alert_error_rate_pct`% and p95 latency > `alert_p95_ms` ms (plus e.g. pod restarts, DB down or disk usage); trigger one on purpose and show

it firing

5.10 Logging

□ Logs from all pods in **one place**, per your `logging_stack`:
- `loki-alloy`: Grafana Loki + Grafana Alloy in the cluster, queried in Grafana;
- `cloud-logging-fluentbit`: Fluent Bit ships pod logs to CloudWatch Logs / Cloud Logging

□ Show a query that follows **one** `request_id` through the logs
□ Retention is configured (say how long, and why)

5.11 CI/CD (GitHub Actions)

□ CI on every PR: lint, tests, build, security gates (5.8)
□ CD on merge to `main`: build → push (SHA tag) → deploy to the cluster
□ Release on tag `v*`: push the semver tag, deploy to "prod" behind a **GitHub Environment with required approval**
□ Documented **rollback**: one command or one workflow run
□ No long-lived cloud keys in GitHub secrets (or, in Learner Lab only, documented short-lived ones)

5.12 Infrastructure as code

□ Terraform creates the network, VMs or cluster, registry, buckets (with lifecycle), firewall rules and secret **placeholders** (values are set outside Terraform or via a variable never committed)
□ `terraform plan` is clean after `apply`; variables and outputs are documented
□ Proof of reproducibility: `terraform destroy` and `apply` again → platform comes back (record it or show it at the defence)
□ State is not in git (remote state in a bucket is a plus)

5.13 Documentation & portfolio

□ `README.md`: what it is, **architecture diagram**, how to deploy from zero, how to roll back, links to dashboards and the demo video
□ At least **4 ADRs** (`docs/adr/0001-….md`): e.g. why k3s, why this logging stack, how secrets reach pods, backup strategy
□ Cost estimate (Pricing Calculator) and what you destroyed at the end
□ **Demo video, 3–5 minutes**, linked in the README

---

6. Suggested milestones (optional)

You can submit any time before the final, but this order works:

Week	Milestone
10	Repo from template, branch protection, variant table, compose works locally
11	CI with lint/tests and security gates; image in the registry
12	Terraform: VPC, VM(s), k3s, registry, buckets
13	App + DB in Kubernetes, roles, secrets from the secret manager, backups + restore
14	Monitoring, alerts, logging, CD with approval and rollback
15	Docs, ADRs, video, report → defend

---

7. Submission and defence

1. **Repository link** (public) — frozen at the commit you name in the report (tag `final`).
2. **Report** — fill in `REPORT_TEMPLATE.md`, export to **PDF**, submit as instructed. Graded by `RUBRIC.md`.
3. **Presentation (~7 min) + oral defence (~8 min)**: show the system live, or the demo video if you already destroyed it; then answer questions about **your** code and decisions. You may be asked to change something live (e.g. roll back, scale, show a log query).

Part	Points
Report (by rubric)	20
Presentation & demo	8
Oral defence	12
Total	**40**

---

8. Academic honesty

- Your variant must be visible everywhere it applies. Someone else's variant = 0 for the report.
- Using documentation, Stack Overflow and AI assistants is allowed; **not being able to explain your own code at the defence is not**. If you can't explain it, the report points for that area are not given.
- Identical repositories, histories or reports → both students are capped at the defence points only.

---

9. FAQ

My Learner Lab session keeps expiring. Expected. Keep everything in Terraform and scripts so you can bring the platform back quickly. Use the self-hosted runner on your VM so CI uses the instance role, not your temporary keys.

Can I use a managed database to save time? No. The self-hosted DB with roles, policy and backups is part of the grade.

Can I use EKS/GKE instead of k3s? Yes, if your `cluster` variant is `managed-k8s-if-credits` and you have the credits; any student may also upgrade from k3s to managed. Grading is the same, and the node failure drill still applies.

Which free tier should I use, and how do I not get billed? See FREE_TIER_GUIDE.md.

My credits ran out. Submit the demo video, Terraform and logs of the working system. For the **orchestration demo only**, a local `kind` / `k3d` cluster is accepted at the defence, with the Orchestration area capped at half points. Everything else must have been shown in the cloud (screenshots/video).

Do I need a domain and HTTPS? No. An IP or the cloud's default DNS name is fine. HTTPS (cert-manager + Let's Encrypt or a cloud certificate) is a plus that can compensate small gaps elsewhere.

How big should the report be? As short as possible, as long as every rubric item has evidence. Link to files and pipeline runs instead of pasting long code.